Fraud Intelligence AI Assistant

FIAA

Capstone Design Document

EY GDS | AI & Data Practice

Capstone Project 2025

Team Members	Role
Darshan J	AI Engineer — Agent pipeline, RAG, LangGraph orchestration
Sabi Qunnar	AI Engineer — Guardrails, Observability, UI, Deployment

Item	Detail
Project name	Fraud Intelligence AI Assistant (FIAA)
Domain	Banking — Fraud Detection & Reporting
Document type	Capstone Design Document
Version	v2.0 — 2025
Stack	LangGraph · Groq · ChromaDB · Tavily · Streamlit · Docker

1. Objective

Fraud Intelligence AI Assistant (FIAA) is an autonomous multi-agent GenAI system that automates fraud incident investigation and reporting in banking. When a suspicious transaction is detected by the bank's fraud rule engine, FIAA automatically researches the case using live web intelligence and institutional knowledge, synthesises a complete fraud incident report with risk score and recommended actions, and delivers it to the analyst's dashboard — all in under 90 seconds with zero manual input.

Core problem: Fraud analysts manually spend 45–90 minutes per case — searching for fraud patterns, checking regulatory deadlines, pulling past cases, and writing reports. The 2-hour SLA for SWIFT recall decisions means slow investigation directly costs money that cannot be recovered.

Core solution: FIAA replaces 45–90 minutes of manual research with a 90-second autonomous agent pipeline. The analyst's job becomes: read the report, click one action button.

2. Goals

Stage 1 — Core pipeline (MVP)

- Webhook receiver — FastAPI endpoint that auto-accepts fraud alerts from bank rule engine
- Multi-agent pipeline — LangGraph Supervisor routing four research agents in sequence
- RAG knowledge base — ChromaDB with past fraud cases, RBI guidelines, response playbooks
- Report generation — Groq Llama 3.3 70B synthesising all context into structured fraud report
- Evaluator quality gate — verifies evidence, score, PII masking, RBI accuracy
- Input and output guardrails — injection detection, PII redaction, hallucination check
- Streamlit dashboard — live agent status, report display, action buttons, download

Stage 2 — Production hardening

- Structured logging — structlog JSON lines with run_id binding and secret redaction
- Prometheus metrics — 7 metrics exposed at /metrics for Grafana scraping
- LangSmith tracing — full LangGraph chain traces with prompt/response replay
- RAGAS evaluation — faithfulness, answer relevance, context recall scored per run
- Docker Compose — 4-service containerised deployment (app, chroma, prometheus, grafana)
- Auto-trigger modes — manual fire, timer stream, random stream for demo

3. Non Goals

- Real bank system integration — FIAA simulates the webhook; no live core banking connection
- Account freezing execution — FIAA recommends the action; analyst executes manually
- SWIFT recall execution — FIAA provides case details; correspondent banking executes
- Real-time Aadhaar/PAN validation — KYC verification out of scope
- Training a custom LLM — uses Groq-hosted Llama 3.3 70B via API, no fine-tuning
- Multi-bank deployment — single bank instance; multi-tenancy is future work
- Mobile application — web-only Streamlit UI for this capstone
- Continuous transaction monitoring — alert-based trigger only

4. Project Architecture Diagram and Flow

4.1 System overview

FIAA follows a hub-and-spoke orchestration pattern. The Supervisor Agent sits at the centre of a LangGraph StateGraph, routing sub-tasks to four specialised research agents. All outputs converge at the Report Agent which synthesises a structured fraud incident report. The Evaluator and RAGAS quality-gate the output simultaneously. High-risk cases route to human approval; low-risk cases auto-close. A reinvestigate loop (dashed arc) sends rejected reports back to the Supervisor.

4.2 End-to-end pipeline — architecture diagram

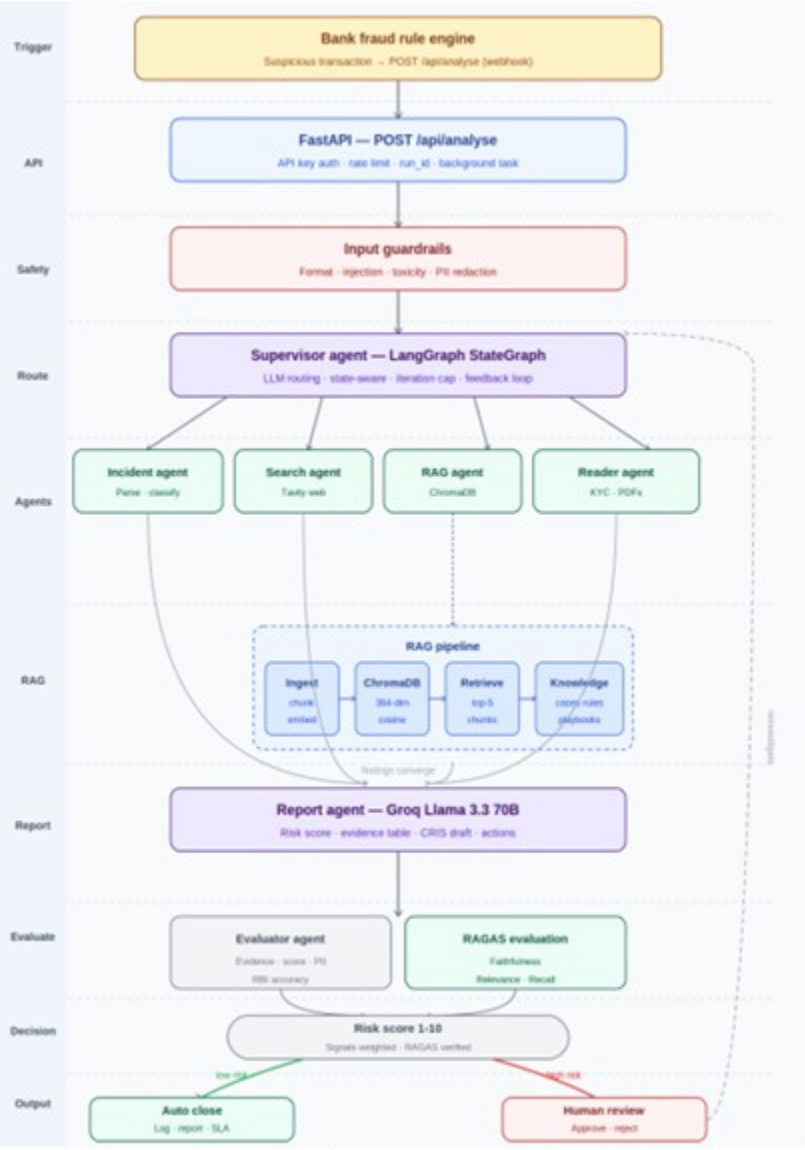

Figure 1 — Full pipeline: webhook trigger → Streamlit dashboard

Figure 1 — Full pipeline: Bank fraud rule engine → webhook → FastAPI → Supervisor → Research agents → RAG pipeline → Report agent → Evaluator + RAGAS → Risk decision → Streamlit dashboard

4.3 Pipeline layer descriptions

Layer	Description
Trigger	Bank fraud rule engine fires POST /api/analyse the moment suspicious transaction detected

Layer	Description
FastAPI	API key auth · rate limiting · run_id assigned · pipeline started as background task
Input guardrails	Format check · injection detection (10 regex) · toxicity filter · PII auto-redaction — all before any LLM call
Supervisor	LLM reads state at each iteration · routes to correct agent · handles evaluator feedback loop
Incident agent	Parses and classifies alert · logs to PostgreSQL/CSV incident database
Search agent	Tavily API — live web: fraud patterns, RBI circulars, SWIFT advisories
RAG agent	ChromaDB cosine search — top-5 past cases, playbooks, RBI guidelines retrieved
Reader agent	PyMuPDF — extracts text from uploaded KYC PDFs and account statements
Report agent	Groq Llama 3.3 70B — synthesises all context into structured fraud report
Evaluator	Evidence grounding · score proportionality · PII check · RBI deadline accuracy
RAGAS	Faithfulness · Answer relevance · Context recall — stored in SQLite per run
Risk decision	Score ≥ 7 → human review · Score < 7 → auto close · Reject → reinvestigate
Output guardrails	PII scan · hallucination detection · citation audit on final report
Dashboard	Streamlit — report, risk score, action buttons, monitoring panel, download

4.4 Supervisor routing logic

The Supervisor is an LLM call — not a hardcoded rule engine. It reads the investigation state at every iteration and decides which agent to call next. Three safety mechanisms prevent infinite loops:

- Iteration cap — force WRITE after 5 iterations regardless of state
- Done-check — never re-call an agent already marked complete in state
- Approval-check — immediately route to END when Evaluator approves

Iteration	State it reads	Decision	Reason
0	Everything missing	→ SEARCH	Need live RBI + fraud corridor data
1	Search done, RAG missing	→ RAG	Need past cases + playbooks
2	Search + RAG done	→ WRITE	Sufficient context gathered
3	Evaluator approved	→ END	Pipeline complete
3	Evaluator rejected	→ WRITE	Revise with evaluator feedback
5	Iteration cap reached	→ WRITE	Force report — prevent infinite loop

4.5 RAG pipeline

Step	Tool	Detail
1. Ingest	rag/ingestor.py	Reads: fraud_playbooks.txt (312 chunks), rbi_guidelines.txt (428 chunks), past_cases.json (107 cases)
2. Chunk	512-token windows	64-token overlap preserves context across chunk boundaries
3. Embed	all-MiniLM-L6-v2 (local)	384-dimension vectors — CPU only, no external API, no customer data leaves machine
4. Store	ChromaDB PersistentClient	Cosine similarity index, persistent across restarts, metadata alongside each vector
5. Retrieve	collection.query()	Top-5 chunks by cosine similarity to current alert embedding
6. Augment	LangGraph state	Chunks injected into Report Agent prompt as structured context

Why ChromaDB: Runs locally (no cloud dependency), persists across restarts (FAISS loses index on Streamlit rerun), stores metadata alongside vectors, and is free. Pinecone sends data to cloud — banking compliance violation.

5. Features

Feature 1 — Automated Alert Ingestion (Webhook)

When the bank's fraud rule engine detects a suspicious transaction it fires a POST request to FIAA's FastAPI endpoint automatically. The pipeline starts without any analyst involvement.

Webhook payload example:

```
POST http://fiaa-server:8001/api/analyse
X-API-Key: fiaa-secret-key-2025
{
  "alert_id":    "FR-2025-8821",
  "account":     "****4729",
  "amount":      487000,
  "destination": "Cayman Islands",
  "signals":     ["velocity_breach", "geo_anomaly", "new_device"],
  "sla_minutes": 120
}
```

Design decision: FastAPI responds to rule engine immediately (< 100ms) with {"status":"accepted"} and runs pipeline as background task. The rule engine is never kept waiting 90 seconds for the report.

Mode	Trigger	What happens
Demo — manual	Analyst clicks Fire alert	Writes to SQLite → pipeline runs → report appears
Demo — timer	Auto-fires every 60 seconds	Hands-free for presentation — no button clicking
Demo — random	Random alert every 45 seconds	Mixed fraud alert stream simulation
Production	Bank rule engine calls webhook	Zero human involvement — alert to report automatic

Feature 2 — Multi-Agent Research Pipeline

Agent	Responsibility	Model	Design decision
Supervisor	LLM routing — reads state, decides next agent	Groq Llama 3.3 70B	LLM not rules — handles alert types never seen before
Incident Agent	Parse, classify alert, log to database	Groq Llama 3.1 8B	Small model sufficient for extraction
Search Agent	Tavily live web — fraud patterns, RBI, SWIFT	Groq Llama 3.1 8B	Small model for synthesis; saves 70B quota
RAG Agent	ChromaDB cosine search — cases, playbooks, rules	Local + Groq	Local embeddings — no customer data to external API
Reader Agent	Extract from uploaded KYC / account PDFs	Groq Llama 3.1 8B	Only runs if analyst uploads a document
Report Agent	Synthesise all context into fraud report	Groq Llama 3.3 70B	Large model reserved here — report quality critical
Evaluator	Quality gate —	Groq Llama	Mirrors EY preparer-reviewer model

Agent	Responsibility	Model	Design decision
	evidence, score, PII, RBI	3.3 70B	

6. Tech Stack

Layer	Package	Version	Design decision
Agent orchestration	LangGraph	0.2+	Typed shared state + conditional routing + parallel fan-out
LLM inference	Groq SDK + LangChain-Groq	latest	500 tok/s throughput. Fallback chain on 429: 70b→8b→gemma2.
Vector database	ChromaDB	0.5+	PersistentClient survives reruns. Local — privacy compliant.
Embeddings	sentence-transformers	2.7.0	all-MiniLM-L6-v2 — local CPU, no API cost, GDPR safe.
Web search	Tavily	0.3+	Pre-extracted content not raw HTML. Relevance-scored.
API layer	FastAPI + uvicorn	0.110+	Async background tasks, rate limiting, Pydantic validation.
UI	Streamlit	1.35+	Live agent cards, monitoring panel, auto-trigger modes.
Logging	structlog	24.1+	JSON lines, run_id binding, secrets redacted automatically.
Metrics	Prometheus + Grafana	0.20+	@track_agent decorator — zero boilerplate in agents.
Tracing	LangSmith + OTel	latest	Full chain traces, prompt replay, token cost. 3 env vars.
RAG evaluation	RAGAS	0.1+	Faithfulness, relevance, recall per run in SQLite.
Deployment	Docker Compose	latest	4 services, single command, shared volumes.

7. Directory Structure

```
fiaa/
├── app.py                      Streamlit UI — live agent dashboard
├── start.py                    Starts Streamlit :8501 + FastAPI :8001
├── requirements.txt
├── .env.example                GROQ_API_KEY, TAVILY_API_KEY, LANGCHAIN_API_KEY
├── docker-compose.yml          4-service deployment
├── Dockerfile
├── prometheus.yml              Scrape config: localhost:8000/metrics
├── api/
│   └── webhook.py              FastAPI — POST /api/analyse
├── graph/
│   ├── state.py                ResearchState TypedDict
│   └── workflow.py             LangGraph StateGraph wiring
├── agents/
│   ├── supervisor.py           LLM routing brain
│   ├── incident_agent.py       Parse, classify, log
│   ├── search_agent.py         Tavily web search
│   ├── rag_agent.py            ChromaDB retrieval
│   ├── reader_agent.py         KYC / PDF extraction
│   ├── report_agent.py         Groq synthesis
│   └── evaluator_agent.py      Quality gate
├── rag/
│   └── ingestor.py             Build ChromaDB KB (run once)
├── guardrails/
│   ├── input_guard.py          Format, injection, toxicity, PII
│   └── output_guard.py         PII scan, hallucination, citations
├── monitoring/
│   ├── logger.py               structlog JSON lines
│   ├── metrics.py              Prometheus + @track_agent
│   └── tracer.py               OpenTelemetry spans
├── configs/
│   ├── settings.py             Pydantic BaseSettings
│   └── prompts.py              All agent system prompts
└── data/
    ├── knowledge_base/
    │   ├── fraud_playbooks.txt   312 chunks
    │   ├── rbi_guidelines.txt    428 chunks
    │   └── past_cases.json       107 cases
    └── sample_alerts/            5 demo scenarios
```

8. Logging

8.1 Structured logging — structlog

All logging uses structlog with JSON output. Every log line carries a run_id that ties all events for one pipeline run together — filter by run_id to see the complete story of one fraud case from webhook to dashboard.

Feature	Implementation	Design decision
JSON lines	structlog.processors.JSONRenderer()	Machine-parseable — ingestible by Loki, Datadog, CloudWatch
run_id binding	structlog.contextvars.bind_contextvars()	Set once per run — all agent lines carry it automatically
Secret redaction	Custom _redact_secrets processor	Scans for api_key/token/secret — replaces with ***REDACTED***
Per-run files	FileHandler(logs/run_{id}.jsonl)	One file per pipeline run — easy per-case post-mortem
Global log file	FileHandler(logs/fiaa.jsonl)	All runs in one file for aggregation and monitoring
Dev format	ConsoleRenderer(colors=True)	Human-readable coloured terminal output during development
Prod format	JSONRenderer to file only	Clean JSON — no colour codes, no console noise

8.2 Events logged per pipeline run

Event	Level	Key fields
pipeline.start	INFO	run_id, alert_id, trigger_len, env
guardrail.pii_redacted	WARNING	run_id, fields_redacted
supervisor.decision	INFO	next, reason, iteration, run_id
agent.success	INFO	agent, duration_s, tokens, run_id
agent.failed	ERROR	agent, error, duration_s, run_id
rag_agent.chunk	INFO	case_id, similarity, source, run_id
evaluator.decision	INFO	approved, score, flags, run_id
pipeline.complete	INFO	iterations, total_tokens, risk_score, duration_s, approved

8.3 Sample log output


```
{"timestamp":"2025-06-19T02:17:00Z","level":"info",
 "event":"pipeline.start","run_id":"a1b2c3d4","alert_id":"FR-2025-8821"}

{"timestamp":"2025-06-19T02:17:00Z","level":"warning",
 "event":"guardrail.pii_redacted","fields":["name","mobile"],"run_id":"a1b2c3d4"}

{"timestamp":"2025-06-19T02:17:01Z","level":"info",
 "event":"supervisor.decision","next":"SEARCH",
 "reason":"Need live RBI + Cayman corridor data","iteration":0,"run_id":"a1b2c3d4"}

{"timestamp":"2025-06-19T02:17:04Z","level":"info",
 "event":"agent.success","agent":"search_agent","duration_s":2.814,"tokens":1240}
```

```
{"timestamp":"2025-06-19T02:17:05Z","level":"info",
 "event":"rag_agent.chunk","case_id":"FR-2024-3341","similarity":0.912}

{"timestamp":"2025-06-19T02:17:09Z","level":"info",
 "event":"agent.success","agent":"report_agent","risk_score":9.2,"tokens":2108}

{"timestamp":"2025-06-19T02:17:11Z","level":"info",
 "event":"pipeline.complete","iterations":3,"total_tokens":4812,
 "approved":true,"duration_s":13.2}
```

8.4 Distributed tracing — LangSmith

Setup: LANGCHAIN_TRACING_V2=true · LANGCHAIN_API_KEY=ls_xxxx · LANGCHAIN_PROJECT=fiaa-fraud — LangGraph picks these up automatically. Zero code changes in agent files.

LangSmith feature	What you can see
Run timeline	Full LangGraph chain — which agent ran, how long, in what order
Prompt inspect	Exact prompt sent to every LLM call — what context Report Agent received
Response inspect	Exact LLM output — spot where report went wrong on failed run
Token cost	Token count and cost per agent per run — find expensive agents
Run replay	Replay any historical run — debug evaluator rejections or wrong risk scores
Cross-run compare	Compare two runs — why did run A score 9.2 and run B score 6.1?

9. Monitoring · Observability · Metrics

9.1 Three-pillar observability stack

Every agent call passes through the @track_agent decorator which emits to three observability pillars simultaneously. Zero monitoring boilerplate exists inside agent code.

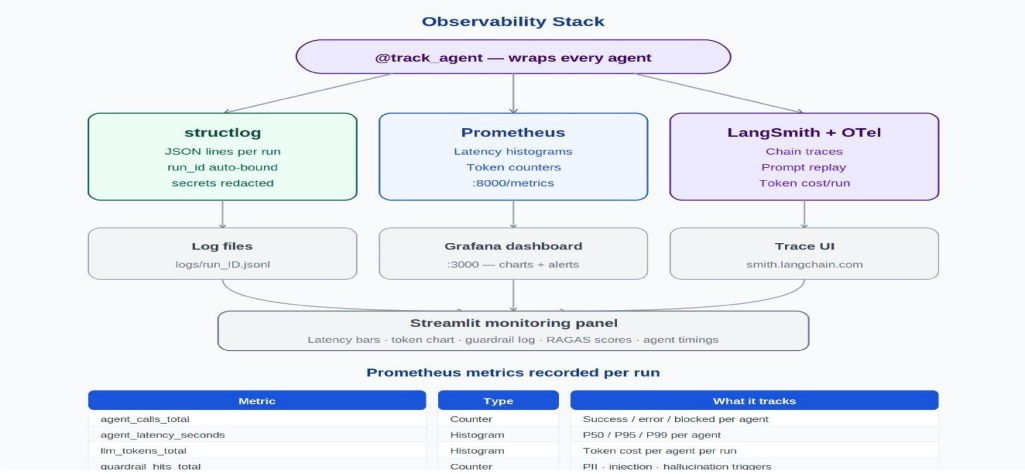

Figure 2 — Three-pillar observability: structlog → log files | Prometheus → Grafana | LangSmith + OTel → trace UI

Pillar	Tool	Output	Purpose
Logging	structlog	logs/run_ID.jsonl	Per-case audit trail — filter by run_id
Metrics	Prometheus + Grafana	localhost:8000/metrics → :3000	Real-time charts, alert rules, capacity planning
Tracing	LangSmith + OTel	smith.langchain.com	Full chain replay, prompt inspection, cost per run

9.2 Prometheus metrics — complete list

Metric	Type	Labels	Purpose	Alert threshold
fiaa_agent_calls_total	Counter	agent, status	Success/error per agent	Error rate > 10% in 5 min
fiaa_agent_latency_seconds	Histogram	agent	P50/P95/P99 per agent	P95 report_agent > 30s
fiaa_llm_tokens_total	Histogram	agent	Token cost breakdown	—
fiaa_guardrail_hits_total	Counter	type, severity	PII, injection, hallucination count	Injection > 5/min
fiaa_pipeline_latency_seconds	Histogram	—	End-to-end pipeline duration	—
fiaa_last_risk_score	Gauge	—	Most recent risk score	—

Metric	Type	Labels	Purpose	Alert threshold
fiaa_active_runs	Gauge	—	Concurrent pipelines now	Active > 10
fiaa_supervisor_iterations	Histogram	—	Routing iterations per run	Mean > 4

9.3 Grafana alert rules

Alert	Condition	Severity
High report latency	P95 fiaa_agent_latency_seconds{agent="report_agent"} > 30s	Warning
Injection spike	rate(fiaa_guardrail_hits_total{type="injection"}[1m]) > 5	Critical
High error rate	rate(fiaa_agent_calls_total{status="error"}[5m]) / rate(...[5m]) > 0.1	Critical
Too many active runs	fiaa_active_runs > 10	Warning
Low RAGAS faithfulness	ragas_faithfulness < 0.7 for 3 consecutive runs	Warning

9.4 RAGAS — RAG pipeline quality evaluation

RAGAS metric	Question it answers	Target	Low score means
Faithfulness	Are all report claims grounded in retrieved RAG chunks?	> 0.85	Report Agent hallucinated — cited case not in retrieved chunks
Answer relevance	Does the report answer the actual fraud alert question?	> 0.80	Response is generic — missed specific alert signals
Context recall	Did RAG Agent retrieve the chunks actually needed?	> 0.75	Right playbook not found — knowledge base needs refresh

10. Stress Testing Scenario

Scenario	Load	Expected behaviour	Pass criteria
Normal load	5 concurrent alerts	All complete < 90s	0% error, P95 < 30s
Peak load	20 concurrent alerts	Queuing, no crashes	< 5% error rate
Injection attack	100 injection attempts/min	All blocked at guardrail	0 reach LLM
RAG empty	KB not built	Graceful degradation, LOW confidence	No crash
Groq rate limit	Primary model hits 429	Fallback to llama-3.1-8b	Pipeline completes
LangSmith down	Tracing endpoint unavailable	Tracing disabled, pipeline continues	0 pipeline failures
ChromaDB restart	Container restarted mid-run	Persistent volume preserves vectors	No data loss
Large alert text	Alert > 5000 chars	Format guardrail blocks at entry	HTTP 400, no pipeline

11. Deployment — Docker Compose

All four services — app, ChromaDB, Prometheus, and Grafana — are defined in a single docker-compose.yml. One command starts the complete system with correct inter-service networking and shared volumes.

Feedback	Mark runs as good/bad — build a quality dataset over time

Grafana alert rules

Alert rule	Condition	Severity
High writer latency	P95 agent_latency_seconds{agent=report_agent} > 30s	Warning
Injection spike	rate(guardrail_hits_total{type=injection}[1m]) > 5	Critical
High error rate	rate(agent_calls_total{status=error}[5m]) / rate(agent_calls_total[5m]) > 0.1	Critical
Too many active runs	active_runs > 10	Warning
Low RAGAS faithfulness	ragas_faithfulness < 0.7 for 3 consecutive runs	Warning

3. Docker Compose Deployment

All four services — app, ChromaDB, Prometheus, and Grafana — are defined in a single docker-compose.yml. One command starts the complete system. Volumes persist data between container restarts. External APIs (Groq, Tavily, LangSmith) require only API keys in the .env file.

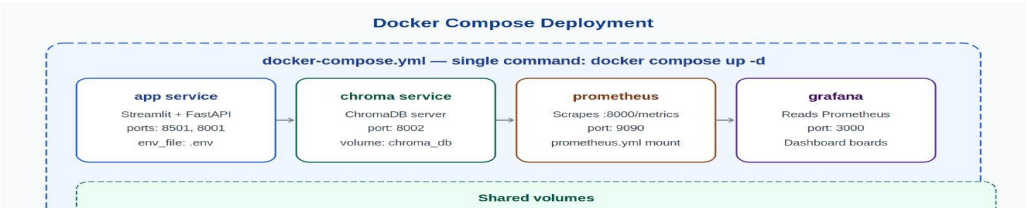

Figure 3 — Four-service Docker Compose: app (Streamlit + FastAPI) · chroma · prometheus · grafana — with shared volumes and external API connections

11.1 docker-compose.yml

```
services:
  app:
    build: .
    ports: ["8501:8501","8001:8001","8000:8000"]
    env_file: .env
    volumes: ["./chroma_db:/app/chroma_db","./logs:/app/logs"]
    depends_on: [chroma]

  chroma:
    image: chromadb/chroma
    ports: ["8002:8000"]
    volumes: ["./chroma_db:/chroma/.chroma"]

  prometheus:
    image: prom/prometheus
    ports: ["9090:9090"]
    volumes: ["./prometheus.yml:/etc/prometheus/prometheus.yml"]

  grafana:
    image: grafana/grafana
    ports: ["3000:3000"]
    environment: ["GF_SECURITY_ADMIN_PASSWORD=fiaa2025"]
```

```
    depends_on: [prometheus]
```

11.2 Service URLs

Service	URL	What you see
Streamlit UI	http://localhost:8501	Live agent dashboard, report, auto-trigger, monitoring
FastAPI webhook	http://localhost:8001/api/analyse	POST endpoint — accepts fraud alert payloads
FastAPI docs	http://localhost:8001/docs	Swagger UI — test webhook from browser
Metrics endpoint	http://localhost:8000/metrics	Raw Prometheus metrics — verify @track_agent
Grafana	http://localhost:3000	Charts, alerts, latency — admin/fiaa2025
LangSmith	https://smith.langchain.com	Full chain traces — project: fiaa-fraud

11.3 Startup sequence

1. Configure: cp .env.example .env — add GROQ_API_KEY, TAVILY_API_KEY, LANGCHAIN_API_KEY
2. Build RAG KB (one time): docker compose run app python -m rag.ingestor
3. Start all services: docker compose up -d
4. Verify dashboard: http://localhost:8501
5. Test webhook: curl -X POST localhost:8001/api/analyse -H "X-API-Key: fiaa-secret-key-2025" -d '{...}'
6. Open Grafana: http://localhost:3000 — add Prometheus data source (http://localhost:9090)

12. Design Decisions

Decision	What was chosen	Why — design rationale
LLM-based Supervisor	LangGraph LLM routing	Handles alert types never seen. Rule engine breaks on edge cases.
Local embeddings	sentence-transformers on CPU	No customer data to external API — banking compliance. No cost. Offline.
Vector database	ChromaDB over FAISS/Pinecone	FAISS: no persistence. Pinecone: cloud = compliance violation. ChromaDB: local, free, metadata built-in.
Model size split	Small 8B for extract, 70B for report	Report quality needs 70B. Extraction does not. Saves token quota.
Webhook trigger	FastAPI POST endpoint	Millisecond latency vs 30s polling. Critical for SWIFT 2-hour window.
Decorator observability	@track_agent wraps agents	Zero boilerplate in agent code. Add/remove monitoring without touching business logic.
RAGAS evaluation	Per-run RAG scoring	Catches knowledge base decay. If faithfulness drops, KB needs refresh.
Docker Compose	Single docker compose up -d	Reproducible. All 4 services start together with correct networking.
Fallback model chain	70b → 8b → gemma2-9b	Groq rate limits on free tier. Ensures pipeline never fails on 429.
Background tasks	FastAPI BackgroundTasks	Rule engine gets 200 OK in < 100ms. Pipeline runs async independently.